

Module 6

SPI — UVM+SV Verification

Learning objectives

- SPI UVM agent: Transaction (SpiTransaction), sequence (SpiSequence), driver (SpiDriver), monitor (S...
- Interface: spi_master_if (clk, rst_n, clk_div_tick, start, data_in, sclk, mosi, cs_n, busy, done) c...
- Mode 0 monitoring: SCLK idle low; capture on rising edge; monitor samples MOSI on each rising SCLK,...

Prerequisites

- Module 1 (spec → RTL), Module 2 (UVM+SV, uvm_smoke), Module 5 (SPI protocol + RTL + basic TB).
- Verilator, Make, C++ compiler, UVM (UVM_HOME or vendored UVM).

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["SPI protocol recap (Mode 0)"]

T2["Toolchain"]

T1 --> T2

T2 --> N["I²C"]

Extend SPI verification to ****UVM+SV**** — SPI agent (transaction, sequence, driver, monitor, scoreboard); run on Verilator.

Overview

- Module 6 builds on Module 5 (SPI protocol + RTL + basic testbench):

How to learn this module

Suggested learning path

- Read the detailed SPI learning guide:
SPI_LEARNING_GUIDE.md — what SPI is, how it works
(signals, M...
- Read the protocols + UVM overview:
LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 5.
How UVM Maps t...

Design architecture

1. SPI DUT (reused from Module 5)

- spi_master + clk_div in
 `module6/examples/spi\_uvm/dut/`.
- spi_master_if connects UVM to
 DUT pins.

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
  subgraph dut["SPI DUT (Module 5)"]
    DIV["clk_div"]
    M["spi_master"]
    DIV --> M
  end
end
```

2. UVM agent structure

- SpiTransaction = one byte;
SpiSequence = directed list;
SpiDriver waits on `done`.
- SpiMonitor samples MOSI on
rising SCLK while `cs\_n` is low
(Mode 0, MSB first).
- SpiScoreboard compares expected
vs `observed\_mosi`.

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

SEQ["SpiSequence"] --> DRV["SpiDriver\nstart, data_in"]

DRV --> DUT["spi_master"]

DUT --> WIRE["sclk, mosi, cs_n"]

WIRE --> MON["SpiMonitor\nsample MOSI @ rising SCLK"]

MON --> SB["SpiScoreboard\nMSB first, Mode 0"]

3. Bus vs DUT checking

- Monitor validates wire behavior;
`done` alone is not sufficient
for sign-off.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SEQ["SpiSequence"] --> DRV["SpiDriver"]

DRV --> DUT["spi_master"]

DUT --> MON["Monitor SCLK/MOSI"]

MON --> SB["Scoreboard"]

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph dut["SPI DUT (Module 5)"]

DIV["clk_div"]

M["spi_master"]

DIV --> M

end

Module 6: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

SEQ["SpiSequence"] --> DRV["SpiDriver\nstart, data_in"]

DRV --> DUT["spi_master"]

DUT --> WIRE["sclk, mosi, cs_n"]

WIRE --> MON["SpiMonitor\nsample MOSI @ rising SCLK"]

MON --> SB["SpiScoreboard\nMSB first, Mode 0"]

Module 6: stimulus, observation, and checking.

SpiTransaction & agent

```
/**
 * Module 6: SPI master UVM test (Verilator)
 *
 * SPI agent: transaction, sequence, driver, monitor, scoreboard.
 * Driver sends bytes via start/data_in; monitor samples MOSI on SCLK
(mode 0);
 * scoreboard checks expected vs observed.
 */

`include "uvm_macros.svh"
import uvm_pkg::*;

`timescale 1ns/1ps

//
-----

-----

// Interface
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module6 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Example directory layout

- dut/: spi_master.v, clk_div.v (same as Module 5)
- test_spi_uvm.sv: Interface, SpiTransaction, SpiSequence, SpiDriver, SpiMonitor, SpiScoreboard, SpiA...

Makefile — UVM SPI sim

```
# Makefile for Module 6 SPI UVM test
#
# Usage:
#   make SIM=verilator TEST=test_spi_uvm
#

SIM ?= verilator
TEST ?= test_spi_uvm

# Prefer externally-set UVM_HOME, but fall back to vendored UVM.
# From module6/examples/spi_uvm, repo root is ../../..
UVM_HOME ?= $(shell echo $$UVM_HOME)
UVM_HOME_FALLBACK := ../../../../tools/uvm-2017/1800.2-2017-1.0
UVM_HOME_FALLBACK_ALT :=
../../../../tools/learn_uvm2017_sv_verilator/tools/uvm-2017/1800.2-2017-1.0
```

Key files to study

Open these in the repo

- ``module6/examples/spi_uvm/dut/spi_master.v`` — reused from Module 5
- ``module6/examples/spi_uvm/test_spi_uvm.sv`` — SpiMonitor samples MOSI on SCLK
- ``docs/LEARNING_GUIDE_PROTOCOLS_AND_UVM.md`` — § 7 SPI UVM mapping

Verification & testing methods

1. Mode 0 in the testbench

- Monitor must use rising-edge capture and falling-edge change — matches CPOL/CPHA=0.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SEQ["SpiSequence"] --> DRV["SpiDriver"]

DRV --> DUT["spi_master"]

DUT --> MON["Monitor SCLK/MOSI"]

MON --> SB["Scoreboard"]

2. Directed regression

- Same byte pattern as UART UVM (0x00 ... 0xFF) for apples-to-apples learning.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SEQ["SpiSequence"] --> DRV["SpiDriver"]

DRV --> DUT["spi_master"]

DUT --> MON["Monitor SCLK/MOSI"]

MON --> SB["Scoreboard"]

3. Extensions

- MISO reads, random data, mode parameters — see [LEARNING_GUIDE § 7] (LEARNING_GUIDE_PROTOCOLS_AND_UV...

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SEQ["SpiSequence"] --> DRV["SpiDriver"]

DRV --> DUT["spi_master"]

DUT --> MON["Monitor SCLK/MOSI"]

MON --> SB["Scoreboard"]

Syllabus topics

1. SPI protocol recap (Mode 0)

- `cs\_n` frames; 8 SCLK cycles per byte; MSB first on MOSI.

2. Toolchain

- ``make SIM=verilator TEST=test_spi_uvm`;`
 ``+UVM_TESTNAME=SpiTest`.`

Hands-on examples

Module 6 self-check

```
$ ./scripts/module6.sh --help
```

Terminal: module6_check

```
[[0;36m===== [[0m
[[0;36mModule 6: Demo commands (SPI UVM example) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version
echo "$UVM_HOME"

# 2) Run the SPI UVM example (inside the example directory):
cd module6/examples/spi_uvm
make SIM=verilator TEST=test_spi_uvm

# 3) Inspect logs and waveforms:
ls obj_dir
# If a VCD is produced:
ls *.vcd

[[1;33m[INFO] [[0m See module6/EXAMPLES.md and module6/examples/spi_uvm/README.md for more details.
```

Example 1: SPI UVM agent

- SPI DUT from Module 5 with UVM driver, monitor, and scoreboard
- Monitor samples MOSI on rising SCLK (Mode 0, MSB first)
- Bus-centric checking vs relying only on the DUT ``done`` flag

Demo: SPI UVM agent

```
$ cd module6/examples/spi_uvm && make SIM=verilator TEST=test_spi_uvm
```

Terminal: ex01_spi_uvm

Success

You should see UVM phases, DRIVER/MONITOR/SCOREBOARD messages, and a scoreboard summary with 5 match

Key concepts

1. Bus-centric verification

- Monitor rebuilds the byte from SCLK/MOSI/CS_N, not only the DUT status flag.

2. Mode 0 in the monitor

- Wait for ``cs_n`` low, then 8 rising edges, MSB first into ``observed_mosi``.

Common pitfalls

Sampling MOSI on wrong edge

- Mistake: Using falling SCLK for Mode 0 capture.
- Correct: Rising-edge samples in SpiMonitor.
- Why: Scoreboard mismatches even when ``done`` pulses.

Ignoring CS framing

- Mistake: Counting SCLK edges across multiple frames.
- Correct: Reset bit index when ``cs_n`` deasserts and a new frame starts.
- Why: Bytes merge or split incorrectly.

Practice & assessment

What you should know

- Describe the SPI UVM agent (transaction, sequence, driver, monitor, scoreboard) and mode 0 monitoring.
- Run `spi_uvm` and interpret UVM output.
- Ready for Module 7: I²C protocol + RTL + basic testbench.

Exercises

- Run spi_uvm
- Trace one transaction
- Change the sequence
- Optional: Compare to UART UVM

Assessment checklist

- Can describe the SPI UVM agent and how it maps to SPI (one byte, mode 0 monitoring).
- Can run `spi_uvm` and interpret UVM output.
- Ready for Module 7: I²C protocol + RTL + basic testbench.

Summary & next steps

- Extend SPI verification to ****UVM+SV**** — SPI agent
(transaction, sequence, driver, monitor, scoreboa...
- Complete module6/CHECKLIST.md
- Review module6/EXAMPLES.md and run each lab
- Next: I²C